GUIDE 10 · QUERIES

Allow a saved query to write data

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=10-allow-a-saved-query-to-write-data. This copy was made on 4 November 2026.

Time: 10 minutes. **You'll end up with:** a saved `INSERT` / `UPDATE` / `DELETE` query that actually runs - and a clear picture of the layered permission checks that have to agree before it does. Every scenario below is verified against a real server: the default-off state, the narrowing-never-widening rule, per-verb restriction, and a write grant scoped to exactly one query.

Why this is off by default

QueryAPIGate runs whatever SQL it's given against your database. A query that reads is relatively safe to expose broadly; a query that writes can corrupt or destroy data if it reaches the wrong caller or runs with the wrong parameters. So every write is off until *something* explicitly turns it on - there is no way to accidentally end up with write access.

Saving a write query needs nothing special

```shell
curl -X POST http://127.0.0.1:5000/api/v1/queries -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "add_note", "sql": "INSERT INTO notes (body) VALUES (:body)",
  "parameters": {"body": {"type": "str"}},
  "connection_name": "shop", "author": "you", "description": "Add a note",
  "publish": true
}'
```

This saves cleanly, no error - **saving** a query with write SQL is never gated. The gate is entirely at *run* time, verified: calling it with no write permission configured anywhere fails immediately, before anything touches the database:

```shell
curl 'http://127.0.0.1:5000/q/add_note?body=hello' -H 'X-API-Key: demo-key'
# {"error": "Only read-only statements (SELECT, WITH, SHOW, DESCRIBE, EXPLAIN) are allowed.
#            Set QUERYAPIGATE_ALLOW_WRITES=1 to lift this restriction."}
```

Turning writes on: the server ceiling

```shell
QUERYAPIGATE_API_KEY=demo-key QUERYAPIGATE_ALLOW_WRITES=1 queryapigate serve
```

Verified: with this set, the admin key's call succeeds and genuinely inserts a row (confirmed with a follow-up `SELECT`). `QUERYAPIGATE_ALLOW_WRITES` is a **server-wide ceiling** - the single switch that makes writes possible on this server at all. Nothing below this point can exceed it, only narrow it further.

A key's own `allow_writes`: narrows the ceiling, never widens it

Verified, this is the important rule: a scoped key created with no `allow_writes` field (defaults to `false`) is still refused, even though the server itself has writes on:

```shell
curl 'http://127.0.0.1:5000/q/add_note?body=blocked' -H 'X-API-Key: <a key with no allow_writes>'
# {"error": "Only read-only statements...", ...} - same error, even though the server allows writes
```

A key with `"allow_writes": true` succeeds instead:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "writer", "connections": ["shop"], "allow_writes": true
}'
curl 'http://127.0.0.1:5000/q/add_note?body=allowed' -H 'X-API-Key: <writer secret>'
# {"message": "No results returned"} - a real insert
```

The rule in one sentence: a write actually happens only when **both** the server-wide setting **and** the calling key's own grant allow it. Either one alone is not enough - a key can never grant itself more than the server allows, and the server being permissive doesn't make every key a writer.

Narrowing which write operations a key may perform

`allowed_write_ops` restricts a writes-enabled key to specific verbs - verified with a key allowed `insert` but not `delete`:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "insert-only", "connections": ["shop"], "allow_writes": true, "allowed_write_ops": ["insert"]
}'

curl 'http://127.0.0.1:5000/q/add_note?body=via-insert-only-key' -H 'X-API-Key: <insert-only secret>'
# {"message": "No results returned"} - insert succeeds

curl 'http://127.0.0.1:5000/q/delete_note?id=1' -H 'X-API-Key: <insert-only secret>'
# {"error": "This API key may only perform these write operations: insert."}
```

Never restricts a read-only statement - only narrows which *write* keywords are permitted once writes are otherwise allowed.

The narrowest option: write access to exactly one named query, nothing else

This is the one worth knowing about even if you never touch `allow_writes` broadly - a key can get write access scoped to a single saved query, with **no** blanket write grant and **no** connection access of its own at all:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "note-writer", "connections": [], "queries": [{"name": "add_note", "allow_writes": true}]
}'
```

Verified, all three outcomes real:

```shell
curl 'http://127.0.0.1:5000/q/add_note?body=per-query-grant' -H 'X-API-Key: <note-writer secret>'
# {"message": "No results returned"} - this one named query, writable

curl 'http://127.0.0.1:5000/q/delete_note?id=1' -H 'X-API-Key: <note-writer secret>'
# {"error": "This API key is not permitted to use the connection 'shop'"} - a different query, refused

curl -X POST http://127.0.0.1:5000/execute_sql -H 'X-API-Key: <note-writer secret>' \
  -H 'Content-Type: application/json' -d '{"sql": "SELECT * FROM notes", "connection_name": "shop"}'
# {"error": "This API key is not permitted to use the connection 'shop'"} - no ad-hoc access at all
```

This is the right shape for "this one integration needs to submit form responses into `submissions`, and should not be able to touch anything else, read or write" - a key that can do exactly one write-shaped thing in the whole system.

What this never does

- **A query's own SQL text decides what "write" means** - there's no separate flag marking a saved query as a write query; `INSERT` / `UPDATE` / `DELETE` /DDL are simply classified as non-read-only by the same guard that checks every statement, saved or ad hoc, every time it runs.
- **No single-statement smuggling** - the same guard that enforces read-only also enforces one statement per call; a write hidden after a semicolon in what looks like a read-only query is caught the same way.
- **MySQL's versioned-comment syntax (`/*! ... */`) is blocked too** when writes aren't allowed - a historical way to hide executable SQL inside what looks like a comment.

Next steps

- Restrict a key to specific tables - `allowed_tables` narrows *what* a key can touch, read or write, independent of everything above.
- Set up your first scoped API key - the full grant reference this guide only used the write-related corner of.